Chapter 1 Diffusion Model

Introduction

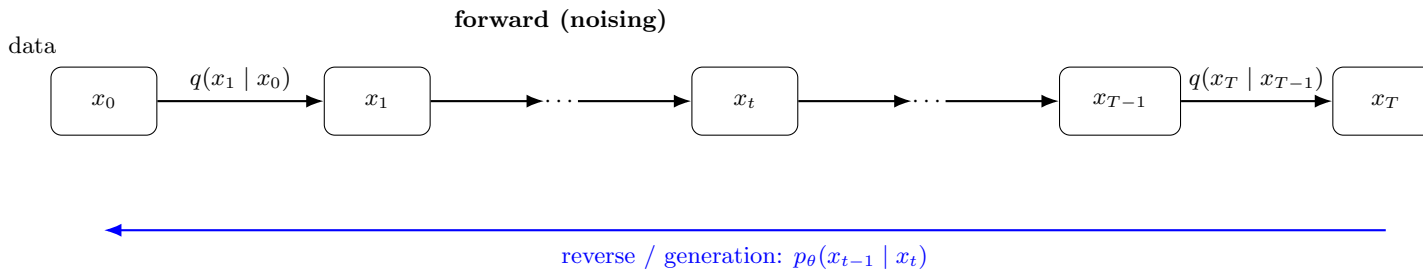
- *Diffusion as iterative generation: noise → data*
- *Markov forward noising process q (parameter-free) and noise schedule*
- *Reverse denoising model p_θ (neural parameterization)*
- *Variational formulation and ELBO decomposition*
- *Closed-form posteriors and the noise-prediction training objective (DDPM)*
- *Sampling algorithm; architectures (U-Net, DiT); extensions (latent diffusion, conditional, flow matching)*

1.1 Why Diffusion? A High-level View

Unlike VAE-style *one-shot* generation (sample a low-dimensional latent $z \sim \mathcal{N}(0, I)$ and decode once), diffusion models generate a sample through *many* small steps. Typically we choose a large horizon T (e.g. $T = 1000$) and start from a pure Gaussian noise $x_T \sim \mathcal{N}(0, I)$, then gradually denoise:

$$x_T \to x_{T-1} \to \cdots \to x_1 \to x_0.$$

Here x_0 lives in the *same* space as the data (e.g. an image vector in $\mathbb{R}^d$), and intermediate states are hidden.



Remark

[Forward vs. reverse time] In diffusion literature, the forward *process means* noising $x_0 \mapsto x_T$, *while the* reverse *process means* denoising $x_T \mapsto x_0$. *The reverse process is the actual* generation *procedure.*

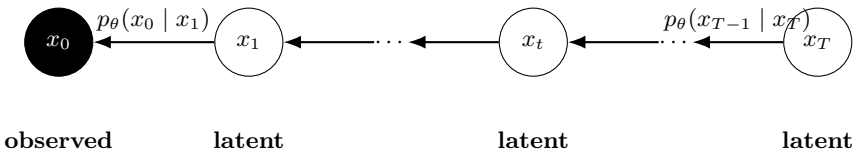
1.2 Graphical Model and Latent Variables

We consider a Markov chain with random variables

$$x_0, x_1, \ldots, x_T \in \mathbb{R}^d,$$

where x_0 is observed (training data) and all $x_1, \ldots, x_T$ are latent. The conditional-independence structure is a simple chain.

Markov chain with hidden states



Note

This is another latent-variable maximum-likelihood problem (similar in spirit to VAE), but with many *latent variables. Directly computing $p_\theta(x_0)$ requires marginalizing all $x_{1:T}$ and is intractable.*

1.3 Model: Reverse (Generative) Process p_θ

We model the reverse denoising chain as

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t), \tag{1.1}$$

with the **prior**

$$p(x_T) = \mathcal{N}(0, I). \tag{1.2}$$

For continuous-valued variables, we typically parameterize each reverse transition as Gaussian:

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\, \Sigma_\theta(x_t, t)\big). \tag{1.3}$$

In practice, both μ_θ and Σ_θ are produced by neural networks that take x_t as input; it is also common to explicitly feed the time index t (or a time embedding) so that the network can distinguish different denoising steps.

1.4 Variational Inference: ELBO for Diffusion

We want to maximize the data log-likelihood:

$$\max_\theta \ \log p_\theta(x_0), \tag{1.4}$$

which is intractable due to latent-variable marginalization. Introduce a variational distribution

$$q(x_{1:T} \mid x_0), \tag{1.5}$$

to approximate the true posterior $p_\theta(x_{1:T} \mid x_0)$. By the standard variational identity,

$$\log p_\theta(x_0) = \mathcal{L}_{\text{ELBO}}(x_0; \theta) + \mathrm{KL}(q(x_{1:T} \mid x_0) \,\|\, p_\theta(x_{1:T} \mid x_0)), \tag{1.6}$$

where the evidence lower bound (ELBO) is

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) := \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} \mid x_0)} \right]. \tag{1.7}$$

Maximizing ELBO is equivalent to maximizing $\log p_\theta(x_0)$ when the variational posterior matches the true posterior.

1.5 Forward (Noising) Process q and Noise Schedule

In DDPM, the variational distribution q is chosen as a **fixed** Markov noising process:

$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}), \tag{1.8}$$

with

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ (1-\alpha_t)I\big), \qquad \alpha_t \in (0,1). \tag{1.9}$$

The sequence $\{\alpha_t\}$ is a **noise schedule** (hyperparameters); typically α_t gradually decreases so that later steps inject more noise.

Remark

[Key DDPM design: q has no trainable parameters] Unlike the VAE encoder $q_\phi(z \mid x)$, the forward chain (1.9) is pre-specified *and contains no learnable parameters. This guarantees that, with a suitable schedule, x_T becomes (approximately) standard Gaussian without needing an explicit KL regularizer.*

1.5.1 Closed-form marginal $q(x_t \mid x_0)$

Define

$$\bar{\alpha}_t := \prod_{s=1}^{t} \alpha_s, \qquad \bar{\alpha}_0 := 1. \tag{1.10}$$

Then one can show by recursion (using Gaussian reparameterization and Gaussian-additivity) that

$$q(x_t \mid x_0) = \mathcal{N}\Big(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)I\Big), \tag{1.11}$$

equivalently,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I). \tag{1.12}$$

Remark

[One-step access to any time t] Equation (1.12) implies we can sample x_t from x_0 in one *shot without simulating the entire chain. This is crucial for efficient training.*

1.5.2 Why $x_T \sim \mathcal{N}(0, I)$ happens

From (1.11), if $\bar{\alpha}_T \approx 0$ (achieved by large T and a decreasing schedule), then

$$q(x_T \mid x_0) \approx \mathcal{N}(0, I),$$

so the end state is close to an isotropic Gaussian noise, matching the reverse-process prior $p(x_T)$.

1.6 A Tractable Posterior: $q(x_{t-1} \mid x_t, x_0)$

The ELBO contains terms involving both q and p_θ. A key trick is to work with the *forward posterior conditioned on* x_0:

$$q(x_{t-1} \mid x_t, x_0). \tag{1.13}$$

Although $q(x_{t-1} \mid x_t)$ is generally intractable, conditioning on x_0 makes it tractable because all relevant terms admit closed forms: $q(x_t \mid x_{t-1})$, $q(x_{t-1} \mid x_0)$, and $q(x_t \mid x_0)$. By Bayes' rule,

$$q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0), \quad \text{with normalization by } q(x_t \mid x_0). \tag{1.14}$$

Theorem 1.1 (DDPM posterior is Gaussian)

Under the Gaussian forward process (1.9), the conditional distribution $q(x_{t-1} \mid x_t, x_0)$ is Gaussian:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t I\big), \tag{1.15}$$

where writing $\beta_t := 1 - \alpha_t$ *and* $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$,

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t, \tag{1.16}$$

and the posterior mean can be expressed as

$$\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t. \tag{1.17}$$

Remark

[Alternative (more useful) mean form: in terms of noise] Using (1.12) (for a fixed t), one may rewrite x_0 *as*

$$x_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\, \varepsilon}{\sqrt{\bar{\alpha}_t}}, \tag{1.18}$$

and substitute into (1.17) to obtain the simplified expression

$$\tilde{\mu}_t(x_t, \varepsilon) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon \right), \tag{1.19}$$

which reveals the intuition: to go from x_t *to the mean of* x_{t-1}*, we "subtract the injected noise direction" with a precise coefficient determined by the schedule.*

1.7 ELBO Decomposition into KL Terms

Expand the ELBO using the Markov factorizations of p_θ and q:

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) = \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log p(x_T) + \sum_{t=1}^{T} \log p_\theta(x_{t-1} \mid x_t) - \sum_{t=1}^{T} \log q(x_t \mid x_{t-1}) \right]. \tag{1.20}$$

Rearranging yields a standard DDPM form:

$$\mathcal{L}_{\text{ELBO}}(x_0; \theta) = -\text{KL}(q(x_T \mid x_0) \,\|\, p(x_T)) + \mathbb{E}_q[\log p_\theta(x_0 \mid x_1)] - \sum_{t=2}^{T} \mathbb{E}_q[\text{KL}(q(x_{t-1} \mid x_t, x_0) \,\|\, p_\theta(x_{t-1} \mid x_t))]. \tag{1.21}$$

Remark

[What is optimized in practice?]

- *The prior KL term is usually treated as a constant (it contains no learnable parameters).*
- *The* $t = 1$ *reconstruction term* $\mathbb{E}_q[\log p_\theta(x_0 \mid x_1)]$ *is sometimes ignored or folded into the* $t = 1$ *case.*
- *The main training signal comes from the sum of KL terms for* $t = 2, \ldots, T$.

1.8 Noise-Prediction Parameterization and the Simple Loss

1.8.1 Predicting noise instead of the mean

We choose $p_\theta(x_{t-1} \mid x_t)$ as Gaussian and parameterize its mean to mirror (1.19):

$$\mu_\theta(x_t, t) := \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right), \tag{1.22}$$

where $\varepsilon_\theta(x_t, t)$ is a neural network (typically U-Net) that predicts the injected noise. For the variance, a common simplification is to fix

$$\Sigma_\theta(x_t, t) = \sigma_t^2 I, \tag{1.23}$$

with σ_t set by a schedule (often related to $\beta_t = 1 - \alpha_t$), rather than learned.

Remark

[Why not learn the variance?] Learning variances introduces log-determinants and matrix inverses in the Gaussian KL closed form, making optimization harder. Empirically, matching the mean *(or equivalently the denoising direction) is most important.*

1.8.2 From KL to a weighted MSE on noise

Because both $q(x_{t-1} \mid x_t, x_0)$ and $p_\theta(x_{t-1} \mid x_t)$ are Gaussians, each KL term in (1.21) has a closed form. Under the parameterization (1.22) and fixed variances, the optimization reduces (up to a time-dependent weight) to matching noises:

$$\mathcal{L}_{\text{simple}}(\theta) := \mathbb{E}_{x_0 \sim p_{\text{data}}} \mathbb{E}_{t \sim \text{Unif}(\{1,\ldots,T\})} \mathbb{E}_{\varepsilon \sim \mathcal{N}(0,I)} \left[\left\| \varepsilon - \varepsilon_\theta\left(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon,\ t\right) \right\|_2^2 \right]. \tag{1.24}$$

The original derivation yields a per-step scaling weight; many implementations drop the weight and still obtain strong performance.

1.9 Training Algorithm (DDPM)

Algorithm 1 DDPM training (noise prediction)

Require: dataset $\{x_0\}$, noise schedule $\{\alpha_t\}_{t=1}^T$, model $\varepsilon_\theta(\cdot,\cdot)$

```
repeat
    Sample a data point x_0 ~ p_data
    Sample a timestep t ~ Unif({1, ..., T})
    Sample noise ε ~ N(0, I)
    Form x_t = √ᾱ_t x_0 + √(1 − ᾱ_t) ε
    Take a gradient step on ||ε − ε_θ(x_t, t)||_2^2
until converged
```

1.10 Sampling Algorithm (Reverse Denoising)

To generate a sample, initialize $x_T \sim \mathcal{N}(0, I)$ and iterate backward:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \tag{1.25}$$

for $t = T, T-1, \ldots, 1$, with the convention $z = 0$ at $t = 1$ (do not re-inject noise at the last step). Substitute (1.22) for μ_θ to obtain the commonly used update:

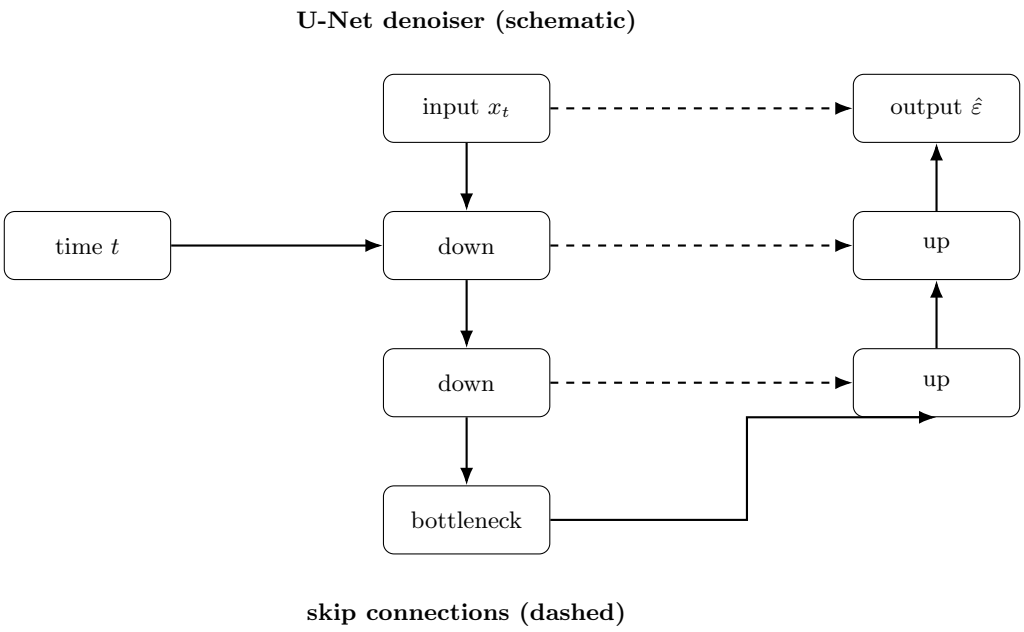
$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\, \varepsilon_\theta(x_t, t) \right) + \sigma_t z. \tag{1.26}$$

Remark

[The "hard part" is the coefficients] Even though training ultimately matches noise, the reverse update uses precise coefficients derived from the probabilistic model. These coefficients are what the ELBO derivation guarantees.

1.11 Network Architecture: U-Net, DiT

The noise network $\varepsilon_\theta(x_t, t)$ maps an input tensor to an output tensor of the *same shape*, which naturally matches U-Net (encoder–decoder with skip connections):



Remark

[DiT] Recent diffusion systems also use transformer backbones (Diffusion Transformer, DiT) as the denoiser. The probabilistic formulation remains similar; the main difference is the architecture used to represent ε_θ.

1.12 Extensions

1.12.1 Latent Diffusion

DDPM operates in pixel space, which can be costly for high-resolution images. Latent diffusion uses a pretrained autoencoder (e.g. a VAE):

- Encode x to a latent $z = \mathrm{Enc}(x)$.
- Run diffusion in latent space to model $p(z)$ or $p(z \mid y)$.
- Decode $\hat{x} = \mathrm{Dec}(z_0)$ back to pixel space.

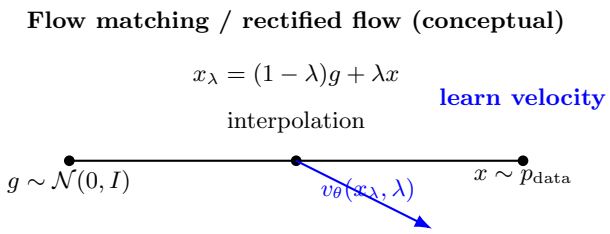
This reduces the dimensionality of the denoiser input/output and accelerates training and sampling.

1.12.2 Conditional generation

To control what is generated (e.g. "cat" vs. "dog"), one trains a conditional model $p_\theta(x \mid y)$ by providing paired data (x_0, y) and feeding y (or a text prompt embedding) into the denoiser. Practical systems often use techniques such as classifier-free guidance.

1.12.3 Flow matching / rectified flow (high-level)

Diffusion can be viewed as learning a trajectory from noise to data. Flow matching methods aim to learn a simpler transport, often a straight-line path, by predicting a velocity field. At a high level, sample an interpolation point between noise g and data x and learn to predict the direction (velocity) pointing from g to x.



Remark

The lecture notes emphasized that many recent state-of-the-art generative models are now based on flow matching / rectified flow formulations.

1.13 Administrative Notes and Q&A (from class)

Remark

[Noise schedule] The schedule $\{\alpha_t\}$ is chosen empirically; it should decrease with t so that $\bar{\alpha}_T$ approaches 0 within a finite horizon (e.g. $T = 1000$), making x_T close to $\mathcal{N}(0, I)$.

Remark

[Why conditioning on x_0 helps in the posterior] Without knowing x_0, many different x_{t-1} could have produced a given x_t under forward noising. Conditioning on x_0 restricts the chain to a specific trajectory, making $q(x_{t-1} \mid x_t, x_0)$ analytically tractable.

Remark

*[Final exam notice] The final exam was announced for **Jan. 7, 2–4pm** in the same classroom, with question style similar to homework (approximately 6–8 problems, difficulty TBD).*